

Thread Synchronization

Objectives:

- Understand the critical section problem and race conditions in concurrent processes.
- Learn to use mutex locks and semaphores (POSIX) for synchronization.
- Apply synchronization concepts to the classic Producer-Consumer problem using shared memory and pipes (already covered in prior labs).
- Detect and prevent deadlock scenarios in multi-threaded programs.

The Concept:

In a multi-threaded program, all threads live in the same house (the process) and share the same variables (global variables). If two threads try to update the same variable at the exact same time without coordination, the data gets corrupted. This is called a **Race Condition**.

Analogy: The Joint Bank Account

Imagine a bank account with **\$100**.

- **Thread A** reads the balance (\$100) and prepares to add \$50.
- At the same split second, **Thread B** reads the balance (\$100) and prepares to add \$20.
- Thread A finishes and writes **\$150**.
- Thread B finishes and writes **\$120**, overwriting Thread A's work.

The final balance is \$120 instead of \$170. You just "lost" \$50 because of a race condition!

Key Terms:

- **Critical Section:** A code segment that accesses shared resources and must not be executed by more than one process simultaneously.
- **Race Condition:** Occurs when the outcome of a program depends on the relative timing of processes/threads.
- **Mutex (Mutual Exclusion Lock):** A locking primitive that ensures only one thread at a time enters the critical section.
- **Semaphore:** An integer variable used for signaling between processes. POSIX semaphores use `sem_wait()` and `sem_post()`.

Mutex Locks

A **mutex** (short for "mutual exclusion") is a lock that allows only one thread to access a shared resource at a time. It is commonly used to protect critical sections in multi-threaded applications.

You need to include the pthread library.

Task A.1 – Race Condition Without Mutex (Observe the Problem)

Create a file *race_no_mutex.c* and enter the following code:

```
#include <stdio.h>
#include <pthread.h>
int counter = 0;
void *increment(void *arg) {
    for (int i = 0; i < 100000; i++) counter++; // NO protection
    return NULL;
}
int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_join(t1, NULL); pthread_join(t2, NULL);
    printf("Counter = %d (expected 200000)\n", counter);
    return 0;
}
```

Compile and run: `gcc race_no_mutex.c -o race_no_mutex -lpthread && ./race_no_mutex`

Notice that the counter value is less than 200000. This is the race condition in action, both threads read and write counter simultaneously without coordination.

Task A.2 – Fix with Mutex (Correct Synchronization)

Create *race_with_mutex.c*. Add the mutex before accessing the counter:

```
#include<stdio.h>
#include<pthread.h>
pthread_mutex_t lock;
int counter = 0;
void* increment_counter(void* arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock); // Lock before the increment
        counter++; // Critical Section
        pthread_mutex_unlock(&lock); // Unlock immediately
    }

    return NULL;}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&lock, NULL); // Initialize the mutex
```

```

pthread_create(&t1, NULL, increment_counter, NULL);
pthread_create(&t2, NULL, increment_counter, NULL);

pthread_join(t1, NULL);
pthread_join(t2, NULL);

printf("Counter = %d (expected 200000)\n", counter);
pthread_mutex_destroy(&lock); // Destroy the mutex
return 0;
}

```

Run the corrected version. The counter should now consistently print 200000.

Steps to Use a Mutex

- Declare a `pthread_mutex_t` variable.
- Initialize the mutex using `pthread_mutex_init()`.
- Lock the mutex using `pthread_mutex_lock()`.
- Unlock the mutex using `pthread_mutex_unlock()`.
- Destroy the mutex using `pthread_mutex_destroy()` when it's no longer needed.

Semaphore:

A **semaphore** is a synchronization mechanism that allows a fixed number of threads to access a shared resource simultaneously.

- **Binary Semaphore:** Works like a mutex (only allows one thread at a time).
- **Counting Semaphore:** Allows multiple threads (up to a specified limit) to access a resource.

Manual for semaphore => https://man7.org/linux/man-pages/man3/sem_init.3.html

How to Use a Semaphore

You need to include `semaphore.h` and `pthread.h`.

Steps to Use a Semaphore

- Declare a `sem_t` variable.
- Initialize it using `sem_init()`.
- Decrement (wait) using `sem_wait()`.
- Increment (signal) using `sem_post()`.
- Destroy using `sem_destroy()` when finished.

Example:

- The semaphore is initialized with 2, so at most two threads can be inside the critical section at the same time.
- `sem_wait()` ensures that only available slots are used.
- `sem_post()` releases the slot for another thread.

```

#include <stdio.h>
#include <pthread.h>

```

```

#include <semaphore.h>
#include <unistd.h>

sem_t sem;

void* worker(void* arg) {
    sem_wait(&sem); // Wait (decrement the semaphore)
    printf("Thread %ld is in critical section\n", (long)arg);
    sleep(1);
    printf("Thread %ld is leaving critical section\n", (long)arg);
    sem_post(&sem); // Signal (increment the semaphore)
    return NULL;
}

int main() {
    pthread_t threads[5];

    sem_init(&sem, 0, 2); // Initialize semaphore to allow 2 threads at a time

    for (long i = 0; i < 5; i++) {
        pthread_create(&threads[i], NULL, worker, (void*)i);
    }

    for (int i = 0; i < 5; i++) {
        pthread_join(threads[i], NULL);
    }

    sem_destroy(&sem); // Destroy semaphore
    return 0;
}

```

Semaphore for Ordering the threads:

So far you used semaphores to LIMIT how many threads enter a section. But semaphores can also be used to CONTROL THE ORDER in which threads execute. This is called the signaling pattern.

The key idea is:

- Initialize the semaphore to 0
- Thread B calls `sem_wait()`; it immediately blocks because value is 0
- Thread A does its work then calls `sem_post()`; this unblocks Thread B
- Now Thread B runs AFTER Thread A, always, guaranteed

Think of it like a traffic signal. Thread B sits at a red light (`sem_wait` on 0). Thread A finishes its work and turns the light green (`sem_post`). Only then does Thread B move.

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

```

```
sem_t sem;

void* threadA(void* arg) {
    printf("Thread A: Doing work first\n");
    sem_post(&sem); // signal Thread B to proceed
    return NULL;
}

void* threadB(void* arg) {
    sem_wait(&sem); // wait until Thread A signals
    printf("Thread B: Running after Thread A\n");
    return NULL;
}

int main() {
    pthread_t a, b;
    sem_init(&sem, 0, 0); // initialized to 0 -- this is the key

    pthread_create(&b, NULL, threadB, NULL); // B created first
    pthread_create(&a, NULL, threadA, NULL); // A created second

    pthread_join(a, NULL);
    pthread_join(b, NULL);

    sem_destroy(&sem);
    return 0;
}
```

Output:

Thread A: Doing work first

Thread B: Running after Thread A